CortexAI LangGraph multi-agent System Documentation

This document explains how LangGraph is implemented inside the Agent Service (Port 8003) to build, orchestrate, and run multi-agent workflows from start to finish.

---

1. What is LangGraph? (For Beginners)

In a basic AI application, you send a prompt directly to a model and get a response.
With **LangGraph**, we create a workflow (a graph) made of **Nodes** (steps) and **Edges** (connections):

1. **Nodes (The Agents)**: Individual functions that perform tasks, like classifying queries, writing code, or holding general conversation.
2. **Edges (The Path)**: Rules that define the order of execution.
3. **State (The Memory)**: A shared storage object passed between nodes. When one node updates the state, the next node reads the updated information.

This allows us to analyze user prompts, decide which specialist agent to run, and process complex operations in structured steps.

---

2. Variables & Functions Dictionary

State Definition

- `agentState` (Root schema in `state.js`):
  - **What it does**: Declares the variables passed between nodes in the graph.
  - **Variables**:
    - `prompt`: Stores the user's input query.
    - `aiResponse`: Stores the final output text generated by the agent.
    - `agent`: Stores the selected agent tag (e.g. `"chat"`, `"coding"`) decided by the

router.
        - `conversationId`: Identifies the active database chat log.

Routing & Nodes

- `router` (node function in `router.js`):
    - **What it does**: Directs the query to the correct specialist agent.
    - **How it works**: Uses the Groq LLM model to read `state.prompt` and evaluate it against rules. It returns a single word (like `"chat"`, `"coding"`, `"search"`) which is written to the state's `agent` variable.
- `getModel` (helper function in `llmmodel.js`):
    - **What it does**: Selects which LLM provider and model to connect to.
    - **How it works**: Returns a Groq instance (`openai/gpt-oss-120b`) for routing, general chat, and search nodes, or a Gemini instance (`gemini-2.5-flash`) for the coding agent.
- `chat` (node function in `chat.agent.js`):
    - **What it does**: Processes general queries.
    - **How it works**: Calls the Groq LLM with a system prompt and `state.prompt` context, returning the answer inside `aiResponse` state.
- `searchagent`, `codingagent`, `pdfgenagent`, `pptgenagent`, `visionnagent` (node stubs inside `agents/` folder):
    - **What they are**: Placeholder node functions prepared to implement specific agent behaviors.

Graph Compilation & Execution

- `workflow` (graph instance in `graph.js`):
    - **What it is**: The instantiated `StateGraph` object.
    - **How it is wired**:
        - `addNode()` registers the node functions (`router`, `chat`, etc.).
        - `addEdge("__start__", "router")` forces the graph to start at the router.
        - `addConditionalEdges()` checks `state.agent` and directs flow to the matching node.
        - `workflow.addEdge("search", "chat")` routes search outputs into the chat node.
        - Other nodes connect straight to `_end_`.

- `graph` (compiled instance in `graph.js`):
  - **How it works**: Compiles the workflow into a runnable instance using `workflow.compile()`.
- `agent` (controller function in `agent.controller.js`):
  - **How it works**: Entry point for API calls. Triggers the graph using `await graph.invoke({ prompt, conversationId })`, reads the returned `aiResponse`, and returns it to the client.

---

3. Global Integration (Where, When, & How LangGraph is Imported and Used)

The LangGraph orchestration module is integrated into the core backend to execute AI flows:

A. Graph Execution and Controller Integration

- **Where**: Imported as `import { graph } from "../graph/graph.js"` inside `agent.controller.js`.
- **When**: Executed inside the `agent` controller endpoint handler (`POST /chat`).
- **How**: Receives prompt and conversation context from the client body, calls the database to log the request, and triggers graph execution with `graph.invoke()`.

B. State Mapping and Compilation

- **Where**: Imported as `import { agentState } from "./state.js"` inside `graph.js`.
- **When**: Runs during Graph compilation on server initialization.
- **How**: Ensures that the `workflow` conforms to the state keys defined in the schema annotation, enabling LangGraph to synchronize variables across different files.

C. LLM Provider Resolution

- **Where**: Imported as `import { getModel } from "../config/llmmodel.js"` inside `router.js` and `chat.agent.js`.
- **When**: Executed dynamically when each node runs and needs an LLM client.
- **How**: Calls `getModel("router")` or `getModel("chat")` to retrieve the correct model

wrapper. The config file `llmmodel.js` uses standard `@langchain` imports to connect to Groq or Google GenAI API services.

---

4. File-to-File LangGraph Execution Workflow

This is the exact sequence of files executed when a prompt is sent to LangGraph:

```
[Trigger] agent.controller.js
   │  ──> Receives prompt and conversationId.
   │  ──> Invokes graph: graph.invoke({ prompt, conversationId }).
   ▼
[Initialization] graph/graph.js
   │  ──> Reads state.js to compile state properties.
   │  ──> Executes start edge, routing control to graph/router.js.
   ▼
[Intent Routing] graph/router.js
   │  ──> Loads router LLM configuration from config/llmmodel.js.
   │  ──> Prompts LLM to select an agent tag.
   │  ──> Writes selected category to state: state.agent = "chat".
   ▼
[Conditional Split] graph/graph.js
   │  ──> Reads state.agent ("chat") and routes control to agents/chat.agent.js node.
   ▼
[Agent Resolution] agents/chat.agent.js
   │  ──> Loads conversation LLM configuration from config/llmmodel.js.
   │  ──> Prompts LLM to generate the final response.
   │  ──> Writes answer to state: state.aiResponse = response.content.
   ▼
[End Node] graph/graph.js
   │  ──> Flows to "_end_" node.
   │  ──> Returns the final completed state object to agent.controller.js.
   ▼
[Output Handback] agent.controller.js
      ──> Extracts result.aiResponse and returns it to the frontend client.
```